AgriPioneer — 耕作放棄地を自動で開拓する草刈りロボット

Physical AI 応用1 講座 最終課題 説明資料 ／ omnicampus アカウント ID: 142210

一言でいうと：1 坪の荒れ地にロボットを置くだけで、 **単眼カメラと CNN で草の地図を作り、最適な経路を計算して刈り切り、 土を測って最適な作物を提案する**までを自動で行うシステムを、 MuJoCo 上で一貫して実装・検証した。

100.0 %	0.781	17.0 m	13 %	3 種
刈り取り率 (205/205 本)	知覚 CNN の検証 IoU	刈り終えるまでの走行距離	全面被覆比の走行距離 削減	系に組み込んだ AI の数

1. 課題の背景と目的

日本の第一次産業は深刻な後継者不足にある一方で、少子高齢化と人口減少により 利用されない土地が増え続けている。この二つは無関係ではなく、 **「農地にできる土地はあるが、整備して作付けする人手がない」** という一つの問題の表と裏である。

そこで本課題では、**ロボットを置くだけで土地が農地になる**という将来像を設定し、 その第一歩として次の 3 つを自動化した。

1. **見る**：区画を見回り、どこにどれだけ草が生えているかを把握する
2. **刈る**：できるだけ短い走行距離で、草をすべて刈り取る
3. **測って提案する**：土壌を計測し、その土地に合った作物を提案する

ゴールは「刈り取りが終わり、**耕耘して作付けできる状態**になること」と **「次に何を植えるべきかが提示されていること」**である。

2. 設定したタスクとシミュレーション環境

2.1 タスク

- 区画：**1 坪 = 1.818 m 角**（面積 3.31 m²）
- 草：**ランダムな位置・草丈**。耕作放棄地を想定し、群生 (草むら) と 散在の混合として生成。群生の中心は最小間隔つきでサンプリングする
- ロボット：芝刈り機型の**差動二輪車**（全長 0.30 m、刈刃幅 0.22 m、 高さ 0.52 m のマストに単眼カメラ）
- 達成条件：区画内の草をすべて刈り取り、土を耕す準備ができること

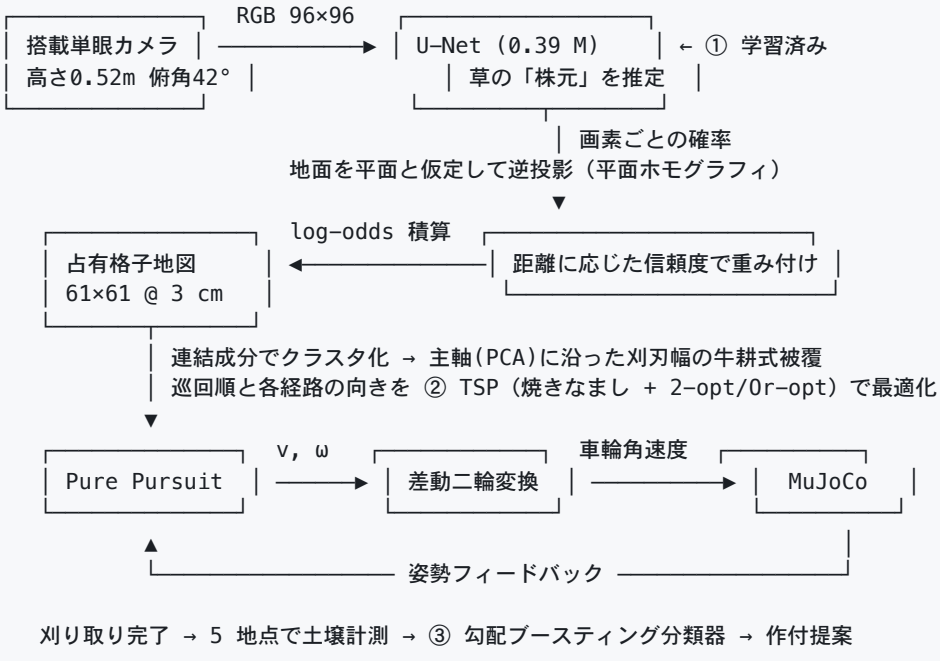


図1: 刈り取り中の画面。左は 3D ビュー（白い点列が計画経路)、 右上が搭載カメラと CNN の推論、右下が構築中の占有格子地図。

2.2 シミュレータ

- **MuJoCo 3.17**（物理 + オフスクリーン描画）
- MJCF は Python から動的生成。草は衝突しない静的 geom とし、 刈り取りは `geom_size` / `geom_pos` / `geom_rgba` の 実行時書き換えで表現（数百本でも実時間の数倍で回る）
- 刈り取り判定は**刈刃の掃引線分と草の距離**で行い、制御周期の離散化による刈り残しを防いでいる

3. システム構成



動作は状態機械 `SURVEY → PLAN → MOW → VERIFY →（刈り残しがあれば MOW へ戻る）→ SOIL → DONE` で構成した。知覚は走行中つねに動いており、刈り取り中も地図が更新され続ける。

4. 組み込んだ AI（3 種）

#	役割	手法	学習データ
①	草の株元セグメンテーション	小型 U-Net（PyTorch, 0.39 M params）	MuJoCo で自動生成した合成画像 4,032 枚
②	刈り取り経路の巡回順最適化	焼きなまし法 + 2-opt / Or-opt（向き付き巡回路）	探索のみ（学習なし）
③	土壌から作物を推薦	HistGradientBoosting 分類器（10 クラス）	営農ルールから生成した合成データ 90,000 件

4.1 ①　アノテーション不要の学習パイプライン（本課題で一番工夫した点）

単に「緑の画素」を拾うだけでは役に立たない。必要なのは **「その草が地面のどこに生えているか」**であって、画像上のシルエットではない。 背の高い草は手前に大きく写り込むため、シルエットをそのまま地面に逆投影すると 実際よりはるかに遠く・広く草があることになってしまう。

そこで教師ラベルを**「草の株元が地面に占める領域」**と定義し、 草の根元の 3 次元座標をカメラへ**順投影して解析的に生成**した。 これにより

- 手作業のアノテーションが**ゼロ**になる（4,032 枚を 48 秒で生成）
- ネットワークは「見えている草の**遠近を解いて**根元の位置を当てる」という 非自明なタスクを解くことになり、出力をそのまま占有格子に積算できる
- 学習データに**刈り取り途中のシーン**（刈り株）を 50 % 混ぜることで、 刈り株を草と誤検出しなくなる = 作業済み判定が自分でできる

加えて、照明方向・環境光・土の色味・草の色味・車体のロール/ピッチを ランダム化（ドメインランダマイゼーション）している。 検証 IoU は **0.7807**（CPU のみで約 30 分の学習）。

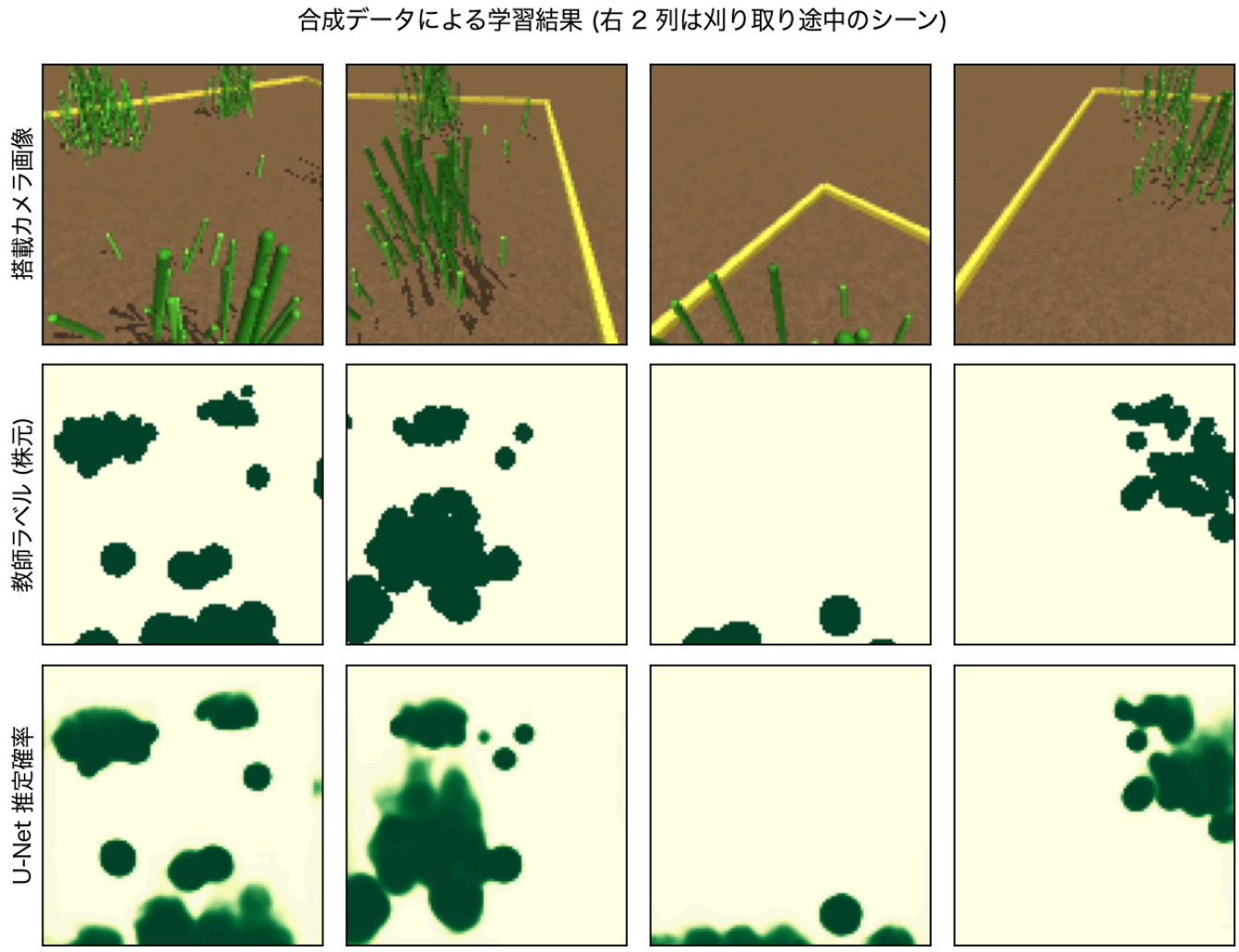


図2: 学習結果。上段が入力、中段が自動生成した教師ラベル、下段が推論結果。 右 2 列は刈り取り途中のシーンで、刈り株を草として検出していないことがわかる。

4.2 ②　被覆経路計画とその最適化

- 占有格子を**刈刃幅ぶん膨張させてから連結成分を取り**、草むらをクラスタに分解する （近接する草むらは 1 回の走行でまとめて刈れるため同一視する）
- 各クラスタの点群に**主成分分析**をかけ、長軸に沿ってレーンを引く。 こうするとレーン本数（=往復回数）が最小になる
- クラスタの**訪問順**と各経路を**どちら向きにたどるか**を同時に最適化する。 これは「向き付き巡回セールスマン問題」であり、最近傍法で初期解を作り、 2-opt（区間反転＋向き反転）・Or-opt（1 要素移動）・向き反転の 3 近傍を 焼きなまし法で探索する
- 追従は **Pure Pursuit**。折れ線のまま追従すると経路点付近で前方注視距離が 縮んで曲率が発散し失速するため、**経路を 5 cm 間隔に密化**してから 弧長で前方注視する実装にした。方位誤差が大きいときはゼロターンに切り替える

4.3 ③　土壌診断から作付提案まで

刈り取り後、区画内 5 地点（巡回順は同じ TSP ソルバで最適化）で pH / EC / 有機物 / 可給態窒素 / 有効態リン酸 / 交換性カリ / 含水率 / 日射 の 8 項目を計測し、中央値を学習済み分類器に入力して上位 3 作物を提示する。

分類器は、10 作物それぞれの適性（適正 pH 帯、要求肥沃度、水分・日照条件、 塩類耐性）を重み付き幾何平均で評価するルールから合成データを作り、 それを勾配ブースティングに蒸留したもの。 ルール出力に対する一致率は **89.4 %** （上位 3 件では **99.6 %**）。

ルールをそのまま使わず学習モデルにしたのは、(1) 欠測やノイズを含む実計測に 滑らかに外挿できる、(2) 確率として順位づけできる、(3) 将来、実地の収量データで そのまま再学習できる、という理由による。

> 土壌の化学特性は MuJoCo では扱えないため、空間相関をもつ合成場で代用している。 「草が密生していた場所ほど有機物と窒素が多い」「有機物が多いほど保水性が高い」 といった相関を入れ、**草の地図と土壌が無関係にならない**ようにした。

本デモでの計測値（5 地点の中央値）

pH	4.75	有効態リン酸	29.08 mg/100g
EC (塩類濃度)	0.20 dS/m	交換性カリ	19.10 mg/100g
有機物含量	3.64 %	体積含水率	29.63 %
可給態窒素	11.40 mg/100g	日射量	11.37 MJ/m2/d

提案された作付

1. **ブルーベリー** **89.0 %**
2. **ジャガイモ** **10.3 %**
3. **ソバ** **0.3 %**

土壌改良の提案

- 主要項目はいずれも適正範囲内. 耕耘後ただちに作付け可能

5. 実験結果

5.1 本デモ（1 坪）の結果

比較対象は、**地図を使わず区画全面を刈刃幅で舐めるベースライン**（一般的なロボット芝刈り機の網羅走行に相当）。同一の草地・同一の制御器で比較した。

指標	ベースライン（全面被覆）	提案手法（知覚駆動）
刈り終えるまでの走行距離	19.58 m	**17.03 m**
刈り終えるまでの所要時間	**64.0 s**	74.9 s
刈り取り率	**100.0 %**	100.0 %
草の分布を把握できたか (地図被覆率)	0.0 %	**100.0 %**



図3: 完了時。すべての草が刈り株になり、土壌診断の結果が提示されている。

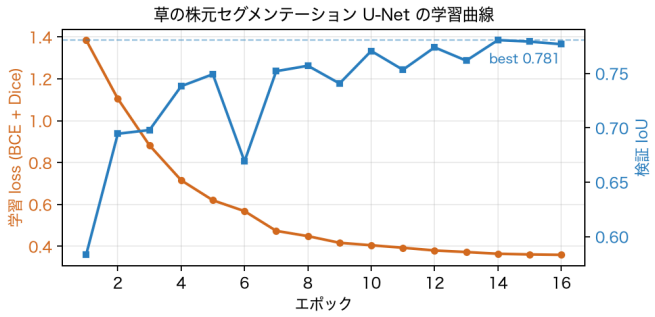


図4: 知覚 CNN の学習曲線。

5.2 どういう条件で有利になるのか（比較実験）

提案手法は「まず見てから刈る」ため、**探査のぶんの走行が上乗せ**される。 一方で草のない場所を走らずに済む。どちらが勝つかをまず単純なモデルで見積もった。

```
全面被覆   L_base ≈ A / w_blade                          (刈刃幅   w_blade = 0.19 m)
提案手法   L_ai   ≈ A / w_cam + A_grass / w_blade        (観測幅   w_cam   = 0.79 m)
→ 有利になる条件:  A_grass / A < 1 − w_blade / w_cam ≈ 76 %
```

これを確かめるため、草の被覆率を 5 % ～ 56 %、区画を 1 ～ 4 坪まで振り、 各条件 3 回ずつ計測した（全 42 走行）。

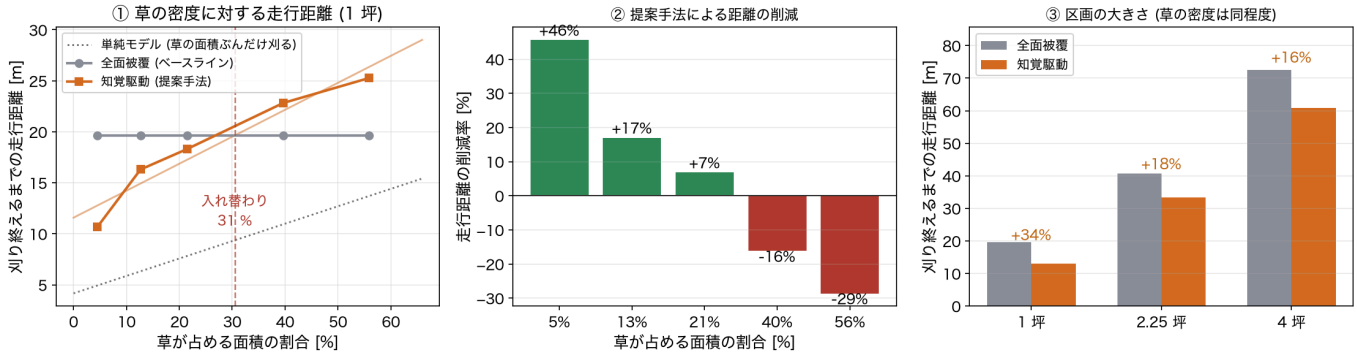


図5: 比較実験の結果（各点は 3 試行の平均）。破線は上の理論値。

草が占める面積	全面被覆	提案手法	距離の削減	刈取率(全面)	刈取率(提案)
4.5 %	19.65 m	10.67 m	**+45.7 %**	100.0 %	100.0 %
12.7 %	19.65 m	16.30 m	**+17.0 %**	100.0 %	100.0 %
21.4 %	19.65 m	18.30 m	**+6.8 %**	100.0 %	100.0 %
39.7 %	19.65 m	22.83 m	**-16.2 %**	100.0 %	100.0 %
55.9 %	19.65 m	25.27 m	**-28.6 %**	100.0 %	100.0 %

区画の大きさ（草の密度は同程度）	全面被覆	提案手法	距離の削減
1 坪	19.65 m	13.02 m	**+33.7 %**
2.25 坪	40.79 m	33.31 m	**+18.4 %**
4 坪	72.59 m	60.80 m	**+16.2 %**

5.3 考察

- **走行距離を決めているのは草の被覆率だった。** 被覆率 5 % では走行距離が **46 % 短く**なる一方、40 % を超えると逆に長くなる。 両者が入れ替わるのは被覆率 **およそ 31 %**。
- **単純モデルの予測（約 76 %）は外れた。** 実測の入れ替わり点はその半分以下である。原因は、刈り取るのが「草そのものの面積」 ではなく **「草むらを刈刃幅ぶん膨張させた領域の外接範囲」**だから。 クラスタの膨張・レーン両端の余裕・クラスタ間の移動により、 実際に舐める面積は草の面積の 3〜5 倍になる。 単純モデルは下限としてしか使えず、設計には実測が要ることが分かった。
- **区画の大きさを変えても優劣はほとんど変わらない**（1 坪 +34 %、4 坪 +16 %)。 上式どおり両者とも面積に比例するためで、 「大きい区画ほど知覚が効くはず」という当初の予想は**誤りだった**。 効いているのは面積ではなく被覆率である。
- **完全性（刈り残しゼロ）は本来トレードオフになる。** 全面被覆は原理的に刈り取り率 100 % を保証するが、 提案手法は知覚の見落としがそのまま刈り残しになる。 初期実装では被覆率の低い条件で 97 % 程度まで落ちた。 そこで**仕上げ巡回で検出しきい値を 0.60 → 0.33 へ段階的に下げ、 最小クラスタサイズも 3 セル → 1 セルに緩める**方式を入れたところ、 **全 42 走行すべてで刈り取り率 100 %** を達成した （しきい値を最初から低くすると誤検出に振り回されて距離が伸びるため、段階的に下げる)。
- 提案手法だけが得られるものとして、**「どこをやり残したか自分で分かる」**点が大きい。 地図があるので作業の検証ができ、そのまま土壌診断の計画にも使える。 全面被覆にはこの情報がない（地図被覆率 0 %)。

6. 工夫した点・つまずいた点

項目	内容
アノテーション不要の 学習データ生成	教師を「草のシルエット」ではなく「**株元の地面占有領域**」と定義し、 3 次元座標の順投影で解析的に生成。手作業ゼロで 4,032 枚、48 秒。 刈り株の混在も自動でラベリングできる。
車輪が滑って 前に進まない問題	MuJoCo は **2 つの geom の摩擦係数を「最大値」で合成する**ため、 低摩擦にしたはずのキャスタが地面側の摩擦に引きずられ、全荷重が キャスタのドラッグに食われていた。`priority` 属性で車輪・キャスタ側の値を 強制採用させて解決（並進速度の追従誤差 75 % → 0 %）。
加速時に駆動輪が浮く	駆動反力で車体がピッチングして「ウィリー」し、駆動輪が浮いて空転していた。重心を前方キャスタ寄りに置き直し、**速度指令にレート制限**を入れて解決。 後方キャスタは通常浮かせておき、転倒防止の保険としてのみ接地させる。
経路点の周りを ぐるぐる回る	折れ線のまま Pure Pursuit すると、経路点に近づくほど前方注視距離が縮んで曲率が発散し失速していた（平均速度 0.09 m/s）。 **経路を 5 cm 間隔に密化**し、弧長で前方注視する本来の実装に直して 0.28 m/s へ。
刈り残しの自己検出	刈刃の掃引領域を地図に「草なし」として書き戻し、刈り取り後に **更新後の地図で再計画**する。残っていれば巡回を繰り返す （しきい値を段階的に下げる）。
地図の作り方	CNN の出力確率をそのまま逆センサモデル（log-odds）として扱い、 **カメラからの距離で重み付け**して積算する。遠方は逆投影誤差が大きいため 重みを落とし、1.25 m より遠い画素は捨てる。
評価の公平性	同一の草地・同一の制御器で比較し、土壌診断の時間は除いた **「刈り終えるまで」**で比較する。理論モデルを立ててから実測し、 予想が外れた点（区画サイズの効果）も含めて記載した。
再現性	レポートの数値はすべて実行結果の JSON から自動で埋め込んでいる。 12 本の回帰テスト（ `pytest` ）で物理・幾何・計画・制御・土壌を検証。

7. 限界と今後

- **土壌は合成場で代用している。**作物適性のルールも一般的な目安を簡略化したもので、 実際の営農判断には地域・品種ごとの検証が必要。実地の土壌分析値と収量データが 得られれば、同じインタフェースのまま分類器を差し替えられる設計にしてある。
- **自己位置はシミュレータの真値を使用。**実機では RTK-GNSS や Visual SLAM が必要で、位置誤差は刈り残しに直結する。
- **草は物理的に干渉しない静的 geom。**草をなぎ倒す力学や、 刈刃にかかる負荷トルクはモデル化していない。
- **今後：**(1) 探査と刈り取りを分けず、観測しながら逐次的に刈る 受動的再計画（anytime coverage）にすれば探査コストを消せる。 (2) 草丈も回帰で推定し、高い草だけ低速で刈るなど刈刃負荷を考慮した速度計画が

できる。 (3) 耕耘・播種アタッチメントまで含めた一連の土地開発へ拡張する。

8. 成果物と再現方法

```
uv venv --python 3.12 .venv && uv pip install --python .venv/bin/python -e .

python scripts/run_mission.py                    # 本デモ（HUD 付き動画 + 指標）
python scripts/run_mission.py --mode baseline    # 比較用ベースライン
python scripts/run_benchmark.py --seeds 3        # 比較実験
pytest -q                                        # 回帰テスト（12 件）

# モデルの再学習（同梱済みなので通常は不要）
python scripts/gen_perception_dataset.py --worlds 112 --poses 36 --res 96
python scripts/train_perception.py --epochs 16   # CPU 10 スレッドで約 30 分
python scripts/train_soil_model.py
```

ROS 2 統合（`ros2_ws/`）も同梱しており、 simulator / perception / planner / controller の 4 ノードをトピックで疎結合にした構成を Docker (ROS 2 Humble) 上で動作確認している。 コアモジュールはスタンドアロン実装とそのまま共有している。

使用ツール: MuJoCo 3.17 / PyTorch / scikit-learn / SciPy / Pillow / ROS 2 Humble。 動画は等速 (1x) 再生・音声なし。